com/files/ and extract the content with *tar zxvf Rpi.GPIO-0.3.1a.tar.gz* and then *cd* to the *RPi.GPIO-0.3.1a* directory, before issuing the following command:

```
pi@raspberrypi /home/pi/RPi.GPIO-0.3.1a $ sudo python setup.py
install
```

This will install the Raspberry GPIO library into Python and you are ready. Write this small script in any text editor, and proceed as follows to connect a red LED across pins 6 and 11 of the RasPi. Pin 11 is the GPIO No 17 (see Figure 1) while Pin 6 is the ground (0 volt). Insert two small sleeves on pins 6 and 11, and then insert the LED directly on to the pins—positive on Pin 11, negative on Pin 6.

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
import time
GPIO.setup(11,GPIO.OUT)
while True:
    GPIO.output(11,True)
    time.sleep(2)
    GPIO.output(11,False)
    time.sleep(2)
```

Save it and run this program as the super user:

```
pi@raspberrypi ~ $ sudo python testgpio.py
```

The LED should be blinking—2 seconds 'on' and 2 seconds 'off'. Terminate the program with Ctrl+C.

Now, the PHP way

First install Apache and PHP on your RasPi, with the following command:

```
pi@raspberrypi ~ $ sudo apt-get install apache2 php5
```

Time to grab a cappuccino and relax while the RasPi installs the packages. To check PHP5, try a simple *info.php* page in */var/www* with the following code:

```
<?php
phpinfo();
?>
```

Now, in the terminal, you can run *php -f /var/www/info.php* or in graphics mode, open the RasPi browser, Midori, and open *http://localhost/info.php* to see if you get the expected pageful of information, which means PHP is working. So far, so good.

To interact with the GPIOs, there is a PHP-GPIO library, which you can download from *https://github.com/pickley/PHP-GPIO/blob/master/GPIO.php* and copy into the */var/www* directory. It's a simple PHP file where all the libraries are included.

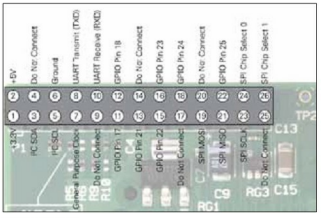

Figure 1: Pi I/O pins

Next, create a PHP script to control the GPIOs with the following code:

```
pi@raspberrypi ~ $ sudo nano /var/www/test.php

<?php
require_once('GPIO.php');
$gpio = new GPIO();
$gpio->setup(17, "out");    //on board connect the LED to pin no
11

while(true) {
        $gpio->output(17, 1);
        usleep(2 * 1000000);    // 2000000 micro Seconds or 2
seconds the LED will be on
       $gpio->output(17, 0);
        usleep(2 * 1000000);  // 2000000 micro seconds off
}
?>
```

Time for testing

At the Pi terminal, run *sudo php -f /var/www/test.php* and if everything goes well, the LED will be blinking with the same frequency as it did for the Python code. If not, go back and check all the connections, and the program.

By: S Bera

The author is a mechanical engineer and presently works as additional general manager at NTPC Limited. An avid lover of open source software, he has deployed OSS both at his home and office, making good use of the LAMP (Linux-Apache-Mysql-PHP) model. He likes to read and travel, and blogs at *http://www.scribd.com/berasomnath*. He can be contacted at *berasomnath@gmail.com*.